CISC 121

// Cycle Rotation

Start

Set numbers = y

Set Rotations = k

If numbers is empty

RETURN an empty list

Make rotated_numbers as a list of zeros,

same length as numbers

For position from 0 to length of numbers - 1

Set new_position = (position + rotations)

// mod length of numbers

Put numbers[position] into

rotated_numbers[new_position]

Return rotated_numbers

End

2) Odd Occurrences in Array

Start

Sort A from Smallest to largest

and call it Sorted_numbers

Set position = 0

While position is before the last number

If Sorted_number[position] is different

from Sorted_numbers[position + 1]

Return Sorted_numbers[position]

Add 2 to position

Return the last number in Sorted_numbers

End

3) FrogJump

Start

Set Start_position = x

Set target_position = y

Set jump_length = D

Set distance_to_go =

target_position - Start_position

Set jumps_needed = number of complete

jumps in distance_to_go

if there is some distance left over

Add 1 to jumps_needed

Return jumps_needed

End

4) Perm Missing Exam

Start

Set numbers = A

Set expected_total = 0

For number from 1 to length of numbers + 1

    Add number to expected_total

Set actual_total = 0

For each number in numbers

    Add number to actual_total

Return expected_total - actual_total

End

5) PermCheck

Start
Sort A from smallest to largest
and call it sorted_numbers

For position from 0 to length of
  sorted_numbers - 1

  Set expected_number = position + 1

  if sorted_numbers[position]
    is not equal to expected_number
      Return 0

Return 1

End